

Tarea 1

Nombre: David Alejandro Díaz Pineda

1. Calcule los errores absoluto y relativo en las aproximaciones de p por p^*

$$E_{abs} = |p^* - p|$$

$$E_{rel} = \frac{|p^* - p|}{|p|}$$

Definimos las funciones para calcular error absoluto y error relativo

```
In [1]: import math
```

```
In [2]: def e_abs(aprox, real):  
        return abs(aprox-real)
```

```
In [3]: def e_rel(aprox, real):  
        return abs(aprox-real)/abs(real)
```

a)

$$p = \pi, p^* = 22/7$$

$$E_{abs} = \left| \frac{22}{7} - \pi \right| = 1.264 * 10^{-3}$$

$$E_{rel} = \frac{\left| \frac{22}{7} - \pi \right|}{|\pi|} = 4.025 * 10^{-4} \approx 0.04\%$$

```
In [4]: print(e_abs(22/7, math.pi))
```

0.0012644892673496777

```
In [5]: print(e_rel(22/7, math.pi))
```

0.0004024994347707008

b)

$$p = \pi, p^* = 3.1416$$

$$E_{abs} = |3.1416 - \pi| = 7.346 * 10^{-6}$$

$$E_{rel} = \frac{|3.1416 - \pi|}{|\pi|} = 2.338 * 10^{-6} \approx 0.0002\%$$

```
In [6]: print(e_abs(3.1416, math.pi))
```

```
7.346410206832132e-06
```

```
In [7]: print(e_rel(math.pi, 3.1416))
```

```
2.338429528530727e-06
```

c)

$$p = e, p^* = 2.718$$

$$E_{abs} = |2.718 - e| = 2.818 * 10^{-4}$$

$$E_{rel} = \frac{|2.718 - e|}{|e|} = 1.037 * 10^{-4} \approx 0.01\%$$

```
In [8]: print(e_abs(2.718, math.e))
```

```
0.0002818284590451192
```

```
In [9]: print(e_rel(2.718, math.e))
```

```
0.00010367889601972718
```

```
In [ ]:
```

d)

$$p = \sqrt{2}, p^* = 1.1414$$

$$E_{abs} = |1.1414 - \sqrt{2}| = 0.273$$

$$E_{rel} = \frac{|1.1414 - \sqrt{2}|}{|\sqrt{2}|} = 0.193 \approx 19.29\%$$

```
In [10]: print(e_abs(1.1414, math.sqrt(2)))
```

```
0.2728135623730952
```

```
In [11]: print(e_rel(1.1414, math.sqrt(2)))
```

```
0.19290831995367475
```

2. Calcule los errores absoluto y relativo en las aproximaciones de p por p^*

a)

$$p = e^{10}, p^* = 22000$$

$$E_{abs} = |22000 - e^{10}| = 26.466$$

$$E_{rel} = \frac{|22000 - e^{10}|}{|e^{10}|} = 1.201 * 10^{-3} \approx 1.20\%$$

```
In [12]: print(e_abs(22000, math.exp(10)))
```

26.465794806717895

```
In [13]: print(e_rel(22000, math.exp(10)))
```

0.0012015452253333286

b)

$$p = 10^\pi, p^* = 1400$$

$$E_{abs} = |1400 - 10^\pi| = 14.544$$

$$E_{rel} = \frac{|1400 - 10^\pi|}{|10^\pi|} = 0.01 \approx 1.05\%$$

```
In [14]: print(e_abs(1400, 10**math.pi))
```

14.544268632989315

```
In [15]: print(e_rel(1400, 10**math.pi))
```

0.010497822704619136

c)

$$p = 8!, p^* = 39900$$

$$E_{abs} = |39900 - 8!| = 420$$

$$E_{rel} = \frac{|39900 - 8!|}{|8!|} = 0.0104 \approx 1.04\%$$

```
In [16]: print(e_abs(39900, math.factorial(8)))
```

420

```
In [17]: print(e_rel(39900, math.factorial(8)))
```

0.010416666666666666

d)

$$p = 9!, p^* = \sqrt{18\pi} \left(\frac{9}{e}\right)^9$$

$$E_{abs} = |\sqrt{18\pi}(\frac{9}{e})^9 - 9!| = 3343.127$$

$$E_{rel} = \frac{|\sqrt{18\pi}(\frac{9}{e})^9 - 9!|}{|9!|} = 9.212 * 10^{-3} \approx 0.921\%$$

```
In [18]: print(e_abs(math.sqrt(18 * math.pi) * ((9 / math.e) ** 9), math.factorial(9)))
3343.1271580516477
```

```
In [19]: print(e_rel(math.sqrt(18 * math.pi) * ((9 / math.e) ** 9), math.factorial(9)))
0.009212762230080598
```

3. Encuentre el intervalo más largo en el que se debe encontrar p^* para aproximarse a p con error relativo máximo de

$$10^{-4}$$

para cada valor de p .

Dado que tenemos

$$E_{rel} = \frac{|p^* - p|}{|p|} \leq 10^{-4}$$

Resolviendo la inecuación tenemos

$$|p^* - p| \leq |p| * 10^{-4}$$

Y por definición de valor absoluto tenemos qué

$$-|p| * 10^{-4} \leq p^* - p \leq |p| * 10^{-4}$$

Sumando p a todos los lados de la inecuación tenemos

$$p - |p| * 10^{-4} \leq p^* \leq p + |p| * 10^{-4}$$

Así, podemos definir la función en python:

```
In [20]: tolerancia = 10**-4
```

```
In [21]: def calcular_intervalo(real):
    margen = abs(real) * tolerancia
    return (f'[{real - margen}, {real + margen}]')
```

a)

$$\pi$$

Teniendo

$$\pi - |\pi|10^{-4} \leq p^* \leq \pi + |\pi|10^{-4}$$

Y sabiendo que

$$|\pi|10^{-4} \approx 0.000314159$$

Tenemos que p^* debe estar en el intervalo $[3.141278, 3.141907]$ para aproximarse a π con un error relativo máximo de 10^{-4} .

```
In [22]: print(calcular_intervalo(math.pi))
```

```
[3.141278494324434, 3.141906812855152]
```

b)

$$e$$

Teniendo

$$e - |e|10^{-4} \leq p^* \leq e + |e|10^{-4}$$

Y sabiendo que

$$|e|10^{-4} \approx 0.00027183$$

Tenemos que p^* debe estar en el intervalo $[2.718010000276199, 2.718553656641891]$ para aproximarse a e con un error relativo máximo de 10^{-4} .

```
In [23]: print(calcular_intervalo(math.e))
```

```
[2.718010000276199, 2.718553656641891]
```

c)

$$\sqrt{2}$$

Teniendo

$$\sqrt{2} - |\sqrt{2}|10^{-4} \leq p^* \leq \sqrt{2} + |\sqrt{2}|10^{-4}$$

Y sabiendo que

$$|\sqrt{2}|10^{-4} \approx 0.000141421$$

Tenemos que p^* debe estar en el intervalo $[1.4140721410168577, 1.4143549837293325]$ para aproximarse a $\sqrt{2}$ con un error relativo máximo de 10^{-4} .

```
In [24]: print(calcular_intervalo(math.sqrt(2)))
```

```
[1.4140721410168577, 1.4143549837293325]
```

d)

$$\sqrt[3]{7}$$

Teniendo

$$\sqrt[3]{7} - |\sqrt[3]{7}|10^{-4} \leq p^* \leq \sqrt[3]{7} + |\sqrt[3]{7}|10^{-4}$$

Y sabiendo que

$$|\sqrt[3]{7}|10^{-4} \approx 0.0001913$$

Tenemos que p^* debe estar en el intervalo

[1.9127398896541117, 1.9131224758906662] para aproximarse a $\sqrt[3]{7}$ con un error relativo máximo de 10^{-4} . -->

```
In [25]: print(calcular_intervalo(math.pow(7, 1/3)))
```

```
[1.9127398896541117, 1.9131224758906662]
```

4. Use la aritmética de redondeo de tres dígitos para realizar lo siguiente. Calcule los errores absoluto y relativo con el valor exacto determinado para por lo menos cinco dígitos.

a)

$$\frac{\frac{13}{14} - \frac{5}{7}}{2e - 5.4}$$

Teniendo como valor exacto:

```
In [26]: exact1 = ((13/14 - 5/7) / (2*math.e - 5.4))  
exact1
```

```
Out[26]: 5.860620417858059
```

Tomamos 5.86062 como referencia ≥ 5 dígitos. Después redondeamos cada uno de los términos de la expresión para después operar.

```
In [27]: term1 = round(13/14, 3)  
term2 = round(5/7, 3)
```

```
term3 = round(2*math.e, 2)
term4 = round(5.4, 3)
```

Calculamos nuestro valor aproximado y lo redondeamos

```
In [28]: num = round(term1 - term2, 3)
den = round(term3 - term4, 3)
valor_aprox = round(num / den, 3)
valor_aprox
```

Out[28]: 5.375

Así, podemos calcular los errores absoluto y relativo

$$E_{abs} \approx 0.486$$

$$E_{rel} \approx 8.286 * 10^{-2} \approx 8.2\%$$

```
In [29]: e_abs(valor_aprox, exact1)
```

Out[29]: 0.4856204178580592

```
In [30]: e_rel(valor_aprox, exact1)
```

Out[30]: 0.08286160563791364

b)

$$-10\pi + 6e - \frac{3}{61}$$

Teniendo como valor exacto:

```
In [31]: exact2 = -10*math.pi + 6*math.e - 3/61
exact2
```

Out[31]: -15.155415893012512

Tomamos -15.15541 como referencia ≥ 5 dígitos. Después redondeamos cada uno de los términos de la expresión para después operar.

- En el primer término tenemos $-10\pi = -31.41592654$, redondeando a 3 cifras significativas nos queda -31.4

```
In [32]: term1 = round(-10*math.pi, 1)
term1
```

Out[32]: -31.4

- En el segundo término tenemos $6e = 16.32$, lo que redondeando a 3 cifras significativas nos queda 16.3

```
In [33]: term2 = round(6*round(math.e,2), 1)
term2
```

Out[33]: 16.3

- Para el tercer término tenemos $-\frac{3}{61} = 0.04918032787$, lo que redondeando a 3 cifras significativas nos da 0.0492

```
In [34]: term3 = round(-3/61, 4)
term3
```

Out[34]: -0.0492

Se calcula el valor aproximado y se redondea

```
In [35]: valor_aprox2 = round(term1 + term2 + term3, 1)
valor_aprox2
```

Out[35]: -15.1

Así, podemos calcular los errores absoluto y relativo

$$E_{abs} \approx 0.0554$$

$$E_{rel} \approx 3.66 * 10^{-3} \approx 0.366\%$$

```
In [36]: e_abs(valor_aprox2, exact2)
```

Out[36]: 0.05541589301251193

```
In [37]: e_rel(valor_aprox2, exact2)
```

Out[37]: 0.003656507574830839

c)

$$\frac{2}{9} * \frac{9}{11}$$

Tomando como valor exacto:

```
In [38]: exact3 = (2/9) * (9/11)
exact3
```

Out[38]: 0.18181818181818182

Tomamos 0.181818 como referencia ≥ 5 dígitos. Después redondeamos cada uno de los términos de la expresión para después operar.

- En la primera operación tenemos $\frac{2}{9} = 0.2222222222$, redondeando a 3 cifras significativas nos queda 0.222


```
In [39]: term1 = round(2/9, 3)
term1
```

Out[39]: 0.222

- En la segunda operación tenemos $\frac{9}{11} = 0.8181818182$, redondeando a 3 cifras significativas nos queda 0.818

```
In [40]: term2 = round(9/11, 3)
term2
```

Out[40]: 0.818

Se calcula el valor aproximado y se redondea

```
In [41]: valor_aprox3 = round(term1 * term2, 3)
valor_aprox3
```

Out[41]: 0.182

Así, podemos calcular los errores absoluto y relativo

$$E_{abs} \approx 1.82 * 10^{-4}$$

$$E_{rel} \approx 9.99 * 10^{-4} \approx 0.01\%$$

```
In [42]: e_abs(valor_aprox3, exact3)
```

Out[42]: 0.0001818181818181719

```
In [43]: e_rel(valor_aprox3, exact3)
```

Out[43]: 0.0009999999999999454

d)

$$\frac{\sqrt{13} + \sqrt{11}}{\sqrt{13} - \sqrt{11}}$$

Tenemos de valor exacto:

```
In [44]: exact4 = (math.sqrt(13) + math.sqrt(11)) / (math.sqrt(13) - math.sqrt(11))
exact4
```

Out[44]: 23.95826074310141

Tomamos 23.95826 como referencia ≥ 5 dígitos. Después redondeamos cada uno de los términos de la expresión para después operar.

- En el primer término tenemos $\sqrt{13} = 3.605551275$, redondeando a 3 cifras significativas nos queda 3.61

```
In [45]: term1 = round(math.sqrt(13), 2)
term1
```

Out[45]: 3.61

- En el segundo término tenemos $\sqrt{11} = 3.31662479$, redondeando a 3 cifras significativas nos queda 3.32

```
In [46]: term2 = round(math.sqrt(11), 2)
term2
```

Out[46]: 3.32

Con esto, en el numerador nos queda $3.61 + 3.32 = 6.93$

```
In [47]: num = round(term1 + term2, 2)
num
```

Out[47]: 6.93

El denominador nos queda $3.61 - 3.32 = 0.29$

```
In [48]: den = round(term1 - term2, 2)
den
```

Out[48]: 0.29

Finalmente, el valor aproximado nos queda

```
In [49]: valor_aprox4 = round(num / den, 1)
valor_aprox4
```

Out[49]: 23.9

Así, podemos calcular los errores absoluto y relativo

$$E_{abs} \approx 0.0582$$

$$E_{rel} \approx 2.43 * 10^{-3} \approx 0.243\%$$

```
In [50]: e_abs(valor_aprox4, exact4)
```

Out[50]: 0.058260743101410384

```
In [51]: e_rel(valor_aprox4, exact4)
```

Out[51]: 0.00243176012341322

5. Los primeros tres términos diferentes a cero de la serie de Maclaurin para la función arcotangente son:

$x - \frac{1}{3}x^3 + \frac{1}{5}x^5$. Calcule los errores absoluto y relativo en las siguientes aproximaciones de π mediante el polinomio en lugar del arcotangente:

Primero definimos la función arcotangente como:

```
In [52]: def arcotangete(x):  
         return x-(1/3)*x**3+(1/5)*x**5
```

a)

$$4\left[\arctan\left(\frac{1}{2}\right) + \arctan\left(\frac{1}{3}\right)\right]$$

Reemplazamos el polinomio en lugar de arctan

```
In [53]: res_a = 4 * (arcotangete(1/2) + arcotangete(1/3))
```

Reemplazando el polinomio por la arcotangente nos queda:

```
In [54]: res_a
```

```
Out[54]: 3.1455761316872426
```

Calculamos el error absoluto y relativo

```
In [55]: print(e_abs(res_a, math.pi))
```

```
0.003983478097449478
```

```
In [56]: print(e_rel(res_a, math.pi))
```

```
0.0012679804598147663
```

Así quedándonos:

$$E_{abs} = 3.983 * 10^{-3}$$

$$E_{rel} = 1.27 * 10^{-3} \approx 0.126\%$$

b)

$$16\arctan\left(\frac{1}{5}\right) - 4\arctan\left(\frac{1}{239}\right)$$

Reemplazamos el polinomio en lugar de la arcotange:

```
In [57]: res_b = 16 * arcotangete(1/5) - 4 * arcotangete(1/239)
```

Lo que nos queda:

```
In [58]: res_b
```

```
Out[58]: 3.1416210293250346
```

Calculamos el error absoluto y relativo:

```
In [59]: print(e_abs(res_b, math.pi))
```

```
2.837573524150372e-05
```

```
In [60]: print(e_rel(res_b, math.pi))
```

```
9.032277055104427e-06
```

Así quedándonos:

$$E_{abs} = 2.837 * 10^{-5}$$

$$E_{rel} = 9.03 * 10^{-6} \approx 0.0009\%$$

6. El número e se puede definir por medio $e = \sum_{i=0}^{\infty} \left(\frac{1}{n!}\right)$ donde $n! = n(n-1)\dots 2.1$ para $n \neq 0$ y $0! = 1$. Calcule los errores absoluto y relativo en la siguiente aproximación de e

Primero definimos una función para la sumatoria:

```
In [61]: def sumatoria_e(inicio, fin):
        suma = 0
        for i in range(inicio, fin+1):
            suma += 1 / math.factorial(i)
        return suma
```

a)

$$\sum_{n=0}^5 \left(\frac{1}{n!}\right)$$

Definimos la aproximación:

```
In [62]: aprox_a = sumatoria_e(0, 5)
         aprox_a
```

```
Out[62]: 2.7166666666666663
```

Calculamos los errores:

```
In [63]: print(e_abs(aprox_a, math.e))
0.0016151617923787498
```

```
In [64]: print(e_rel(aprox_a, math.e))
0.0005941848175817597
```

Así quedándonos:

$$E_{abs} = 1.615 * 10^{-3}$$

$$E_{rel} = 5.942 * 10^{-4} \approx 0.05\%$$

b)

$$\sum_{n=0}^{10} \left(\frac{1}{n!} \right)$$

Definimos la aproximación:

```
In [65]: aprox_b = sumatoria_e(0, 10)
         aprox_b
```

```
Out[65]: 2.7182818011463845
```

Calculamos los errores:

```
In [66]: print(e_abs(aprox_b, math.e))
2.7312660577649694e-08
```

```
In [67]: print(e_rel(aprox_b, math.e))
1.0047766310211053e-08
```

Así quedándonos:

$$E_{abs} = 2.731 * 10^{-8}$$

$$E_{rel} = 1.004 * 10^{-8} \approx 0.000001\%$$

7. Suponga que dos puntos (x_0, y_0) y (x_1, y_1) se encuentran en una línea recta con $y_1 \neq y_0$. Existen dos fórmulas para encontrar la intersección x de la línea:

$$(1)x = \frac{x_0y_1 - x_1y_0}{y_1 - y_0}$$

y

$$(2)x = x_0 - \frac{(x_1 - x_0)y_0}{y_1 - y_0}$$

a) Use los datos $(x_0, y_0) = (1.31, 3.24)$ y $(x_1, y_1) = (1.93, 5.76)$ y la aritmética de redondeo de tres dígitos para calcular la intersección con x de ambas maneras. ¿Cuál método es mejor y por qué?

Primero definimos los puntos

```
In [68]: x0, y0 = 1.31, 3.24
         x1, y1 = 1.93, 5.76
```

Definimos ambas formulas para calcular la intersección x

- Fórmula 1

```
In [69]: def formula1(x0, y0, x1, y1):
         if y1 != y0:
             return (x0 * y1 - x1 * y0) / (y1 - y0)
         return ZeroDivisionError
```

- Fórmula 2

```
In [70]: def formula2(x0, y0, x1, y1):
         if y1 != y0:
             return x0 - ((x1 - x0) * y0) / (y1 - y0)
         return ZeroDivisionError
```

Usamos cualquiera de las 2 para definir nuestro valor real

```
In [71]: valor_real = formula1(x0, y0, x1, y1)
         valor_real
```

```
Out[71]: 0.5128571428571429
```

Aplicamos la aritmética de redondeo de 3 dígitos para cada operación en cada manera

- Fórmula 1:

En la primera fórmula, empezando por el numerador, de primer término tenemos $x_0 y_1 = 7.5456$, redondeando a 3 cifras significativas tenemos 7.55

```
In [72]: term1 = round(x0 * y1, 2)
term1
```

Out[72]: 7.55

En el segundo término tenemos $x_1 y_0 = 6.2532$, lo que redondeando a 3 cifras significativas queda 6.25

```
In [73]: term2 = round(x1 * y0, 2)
term2
```

Out[73]: 6.25

Como numerador queda

```
In [74]: numerador = round(term1 - term2, 3)
numerador
```

Out[74]: 1.3

Siguiendo con el denominador, el único término queda

```
In [75]: denominador = round(y1 - y0, 2)
denominador
```

Out[75]: 2.52

```
In [76]: interseccion1 = round(numerador / denominador, 3)
interseccion1
```

Out[76]: 0.516

Quedándonos como interseccion

$$x = 0.516$$

- Fórmula 2

Empezando con el numerador, como primer término tenemos $x_1 - x_0 = 0.62$

```
In [77]: term1 = round(x1 - x0, 3)
term1
```

Out[77]: 0.62

Como numerador resultante $(x_1 - x_0) y_0 = 2.0088$, redondeando queda 2.01

```
In [78]: numerador = round(term1*y0, 2)
numerador
```

Out[78]: 2.01

El denominador es el mismo, entonces calculamos el término completo

```
In [79]: term_completo = round(numerador / denominador, 3)
term_completo
```

Out[79]: 0.798

Calculamos la resta final para saber la intersección, quedando

$$x = 0.512$$

```
In [80]: interseccion2 = round(x0 - term_completo, 3)
interseccion2
```

Out[80]: 0.512

Calculamos los errores para cada método

$$E_{abs1} = 3.14 * 10^{-3}$$

$$E_{rel1} = 6.13 * 10^{-3} \approx 0.6128\%$$

```
In [81]: print(e_abs(interseccion1, valor_real))
```

0.003142857142857114

```
In [82]: print(e_rel(interseccion1, valor_real))
```

0.006128133704735319

$$E_{abs2} = 8.57 * 10^{-4}$$

$$E_{rel2} = 1.67 * 10^{-3} \approx 0.1671\%$$

```
In [83]: print(e_abs(interseccion2, valor_real))
```

0.0008571428571428896

```
In [84]: print(e_rel(interseccion2, valor_real))
```

0.0016713091922006204

El método más efectivo es el segundo método porque presenta un menor error relativo